

TANGO: Temporal-Spatial Co-Scheduling for Multi-SLO LLM Serving

Abstract—Online Large Language Model (LLM) serving faces the challenge of delivering high throughput while satisfying heterogeneous latency Service-Level Objectives (SLOs). Each request undergoes a prefill phase to produce the first token, followed by a decode phase that generates subsequent tokens, giving rise to two latency metrics: time-to-first-token (TTFT) and time-per-output-token (TPOT). However, existing LLM serving systems struggle to adapt resource allocation to the varying urgency of requests across prefill and decode phases under heterogeneous SLOs. We therefore propose TANGO, a temporal-spatial co-scheduling framework for Multi-SLO LLM serving. TANGO comprises a deadline-aware request dispatcher and a step-time-model-guided token allocator. The dispatcher prioritizes requests based on phase-aware SLO deadlines, while the token allocator regulates per-step workloads using predicted execution time. Together, they coordinate request ordering and batch composition to improve first-token responsiveness and inter-token timeliness. Experimental results show that TANGO improves overall SLO attainment by **58.72%**, and achieves **1.52×** higher goodput on average than state-of-the-art systems.

I. INTRODUCTION

Online large language model (LLM) services consolidate requests from multiple applications on shared model instances to improve GPU utilization and reduce serving costs. These applications impose distinct latency requirements: some requests are sensitive to the delay before the first output token, as in interactive chat, whereas others are sensitive to the delay between consecutive output tokens, as in real-time translation. These latency requirements are associated with two phases of LLM inference: during *prefill*, the model processes the prompt and generates the first output token; during autoregressive *decode*, it generates subsequent tokens one at a time. Time to first token (TTFT) measures the delay from request arrival until the first output token is returned, while time per output token (TPOT) measures the average delay per subsequent token. Online serving systems must therefore maintain high throughput while meeting application-specific TTFT and TPOT service-level objectives (SLOs).

Facing such a scenario, LLM serving aims to improve goodput by satisfying both TTFT and TPOT SLOs for requests with heterogeneous latency requirements. To support this goal, serving systems combine continuous batching, which dynamically admits new requests into the active batch between iterations, with chunked prefill, which divides long prompts into smaller segments that can be processed alongside decoding requests. This combination improves GPU utilization and serving efficiency and is adopted by state-of-the-art inference engines such as vLLM and SGLang.

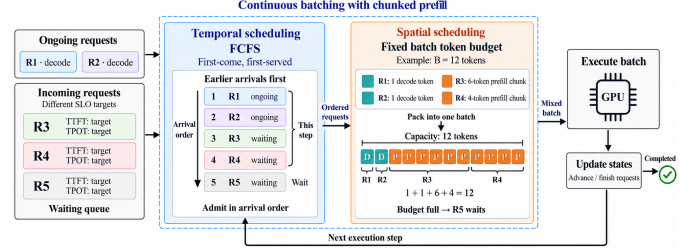


Fig. 1. Temporal and spatial scheduling in LLM serving.

As shown in Fig. 1, incoming requests first enter the engine’s waiting queue. At each execution step, the scheduler selects waiting requests to run alongside ongoing requests. It then allocates the step’s token budget among these requests to form a batch, specifying the sizes of prefill chunks and the number of decode tokens to include. The engine then executes the batch, updates request states, and repeats this process for the next step. We abstract this process into two coupled scheduling dimensions: *temporal scheduling*, which determines request execution priority and admission, and *spatial scheduling*, which determines token allocation within each batch. These two decisions jointly shape request queueing delay and step duration, thereby affecting TTFT and TPOT.

Existing throughput-oriented engines commonly use first-come-first-served (FCFS) admission for temporal scheduling and a fixed maximum token budget for spatial scheduling, without adapting these policies to the heterogeneous SLOs of individual requests. To assess the impact of these policies, we evaluate the default FCFS scheduler on the Qwen3-8B model using a workload with heterogeneous TTFT and TPOT requirements. Among requests with tight TTFT SLOs, only **aa.bb%** satisfy both SLOs, with TTFT violations accounting for most failures. For requests with tight TPOT SLOs, joint SLO attainment is similarly low at **aa.bb%**, with TPOT violations as the main cause. Across the workload, only **aa.bb%** of requests satisfy both TTFT and TPOT SLOs. This low joint attainment highlights the limitations of SLO-unaware scheduling under heterogeneous latency requirements.

First, SLO-unaware temporal scheduling leads to an admission order that does not reflect request urgency. Under FCFS, a request with a tight TTFT objective can wait behind earlier requests with ample latency slack. The urgent request may exhaust its TTFT budget in the queue, even though the earlier requests could tolerate being served later. In our workload, queueing accounts for **aa.bb%** of TTFT on average. FCFS

therefore misses the opportunity to defer less urgent requests and use their slack to reduce queueing delays for requests with tighter SLOs.

Second, existing spatial scheduling policies allocate tokens under a fixed per-step budget without considering the TPOT requirements of active decode requests. Because decoding requests must wait for the entire batch to finish before receiving their next tokens, admitting more prefill work can prolong their inter-token delays. With 2,048 decode tokens fixed, increasing prefill work from zero to 2,048 tokens raises mean step time from **aa.bb ms** to **aa.bb ms**. Moreover, the same total token count can yield different step times: under a 4,096-token budget, pure-prefill and pure-decode steps both take approximately **aa.bb ms**, whereas an evenly mixed step takes **aa.bb ms**. A fixed token budget therefore cannot reliably bound step duration or adapt prefill allocation to the latency tolerance of decoding requests.

Addressing these inefficiencies requires coordinating request-level urgency with batch-level execution time. First, temporal scheduling involves deciding which waiting requests to admit and which ongoing requests can be deferred under resource contention. These requests are governed by different active SLOs and have consumed different portions of their latency budgets, so determining their execution order hinges on a consistent comparison of urgency across phases. Second, spatial scheduling couples the latency requirements of prefill and decode requests through token allocation within a shared batch. Allocating more tokens to prefill can advance requests with urgent TTFT objectives but also prolong the wait for decoding requests, whereas overly restricting prefill can leave those urgent requests waiting. Balancing these competing demands requires accurately estimating step time under varying prefill/decode compositions and execution contexts, then using that estimate to guide per-step token allocation.

We therefore propose TANGO, a multi-SLO LLM serving system that jointly controls temporal request dispatch and spatial token allocation. TANGO aims to improve goodput while satisfying heterogeneous TTFT and TPOT requirements. It comprises a deadline-aware request dispatcher and a step-time-model-guided token allocator. The deadline-aware request dispatcher assigns phase-aware deadlines according to each request’s active SLO and uses them to guide admission and preemption, prioritizing requests with tighter latency budgets. The step-time-model-guided token allocator uses a step-time model to predict the execution time of mixed prefill/decode batches and adjusts their prefill token budgets according to the remaining latency slack of decoding requests.

This paper makes three major contributions:

- **Investigating scheduling inefficiencies in multi-SLO LLM serving.** The analysis motivates us to account for heterogeneous request urgency and mixed prefill/decode interference through joint control of request dispatch and token allocation.
- **The design of a deadline-aware request dispatcher that prioritizes requests across inference phases.** By mapping each request’s active SLO to a comparable

TABLE I
COMPARISON OF TANGO WITH REPRESENTATIVE SERVING SYSTEMS.

	Continuous Batching	Heterogeneous SLOs	Runtime Reprioritization	Predictive Token Budgeting
vLLM	✓			
SOLA	✓		✓	
JITServe	✓	✓	✓	
TANGO	✓	✓	✓	✓

deadline, it guides request admission and preemption, mitigating temporal inefficiency caused by SLO-unaware queueing.

- **The design of a step-time-model-guided token allocator that adapts batch composition to latency constraints.** Based on predicted step time, it converts the remaining TPOT slack into a prefill token budget, mitigating mixed prefill/decode interference while preserving progress for requests with tight TTFT requirements.

We evaluate TANGO against vLLM, SOLA, and JITServe using Llama-3.1-8B, Qwen3-14B, and Qwen3-32B under chat-bot, mathematical-reasoning, and code-generation workloads. Compared with state-of-the-art systems, TANGO improves overall SLO attainment by **aa.bb%** and **aa.bb%**, and achieves up to **a.bb×** the goodput.

II. RELATED WORK

LLM Serving Systems. Recent studies have developed efficient LLM serving systems that improve inference throughput and latency through scheduling and memory optimizations. Orca introduced iteration-level scheduling, allowing requests to enter and leave a batch between decoding iterations to improve both utilization and responsiveness. vLLM further improved serving throughput through PagedAttention, which reduces KV-cache fragmentation and enables flexible memory sharing across requests. SGLang exploits prefix sharing and structured execution to accelerate multi-call LLM workloads. These systems establish continuous batching and efficient memory management as the foundation of modern LLM serving. However, throughput-oriented schedulers commonly use an SLO-agnostic request order and a fixed upper bound on the number of tokens processed in each iteration, without adapting either decision to the heterogeneous TTFT and TPOT requirements of individual requests.

Prefill–Decode Scheduling. The prefill and decode phases exhibit distinct execution characteristics: prefill processes many prompt tokens in parallel, whereas decode repeatedly processes one token per request. Mixing the two phases improves GPU utilization but can prolong an iteration and delay the next tokens of ongoing requests. Sarathi-Serve mitigates this interference through chunked prefill and stall-free batching, using decode-maximal batches to improve serving capacity under tail-latency constraints. DistServe instead places prefill and decode on different GPUs and separately optimizes their resource allocation and parallelism under TTFT and TPOT constraints. These approaches regulate interference through configured chunk sizes, phase-level batching policies, or resource separation. They do not directly translate the

remaining latency slack of active requests into an iteration-specific token budget for a colocated mixed batch.

SLO-Aware LLM Serving. Recent systems explicitly optimize the fraction of requests satisfying latency SLOs rather than raw throughput alone. SOLA formulates a fine-grained scheduling space that adjusts both request execution order and iteration workload according to request and system states, thereby balancing TTFT and TPOT. JITServe estimates uncertain request characteristics and allocates each request sufficient serving bandwidth while grouping compatible requests to improve batch efficiency and goodput. However, existing systems do not adjust the per-step token allocation within a mixed batch according to the request-specific TTFT and TPOT slack. Table I compares TANGO with representative serving systems.

III. BACKGROUND AND MOTIVATION

A. Multi-SLO LLM Serving

LLM Inference and Latency SLOs. An online LLM serving engine processes each request in two phases. During *prefill*, the engine processes all prompt tokens in parallel, constructs their key-value (KV) caches, and produces the first output token. During *decode*, it reuses the KV caches and autoregressively generates subsequent output tokens, typically one token per request in each iteration. The two phases expose different forms of latency and corresponding service-level objectives (SLOs): a request must first wait for its prompt to be processed, and must then make timely progress over a sequence of decoding iterations. Let request r_i arrive at time a_i and generate n_i output tokens, where $t_{i,j}$ denotes the time at which its j -th output token is returned. Its time to first token (TTFT) and time per output token (TPOT) are defined as

$$\begin{aligned} \text{TTFT}_i &= t_{i,1} - a_i, \\ \text{TPOT}_i &= \frac{1}{n_i - 1} \sum_{j=2}^{n_i} (t_{i,j} - t_{i,j-1}), \quad n_i > 1. \end{aligned} \quad (1)$$

TTFT captures the queuing and prefill delay before a response begins, whereas TPOT captures the average interval between consecutive tokens after generation starts. Each request is associated with a pair of latency objectives, denoted by S_i^{TTFT} and S_i^{TPOT} . Request r_i satisfies its SLOs only if $\text{TTFT}_i \leq S_i^{\text{TTFT}}$ and $\text{TPOT}_i \leq S_i^{\text{TPOT}}$. In this work, *Multi-SLO* refers both to these phase-specific objectives and to their heterogeneous values across requests and applications. Accordingly, serving efficiency is measured by goodput—the rate of requests that satisfy both objectives—rather than by raw request or token throughput alone.

Continuous Batching and Chunked Prefill. In static batching, a newly arrived request cannot join an executing batch until every request in that batch finishes, which wastes GPU capacity as requests complete at different times. Continuous batching instead updates the active batch at iteration boundaries. After each iteration, completed requests leave, unfinished decode requests remain active, and waiting requests

can be admitted into the released capacity. The batch composition can thus evolve throughout generation, improving GPU utilization while allowing the scheduler to reconsider which requests should run at every step.

A prefill request may contain hundreds or thousands of prompt tokens, whereas an active decode request contributes only its next token to an iteration. Scheduling an entire prompt at once can therefore produce a long iteration and delay token generation for the decode requests in the same batch. Chunked prefill divides the prompt into smaller chunks that may be processed over multiple iterations. This enables prefill chunks and decode tokens to be colocated in one batch, while exposing the prefill chunk size as a controllable scheduling variable. Specifically, let $\mathcal{P}_k$ and $\mathcal{D}_k$ denote the prefill and decode requests selected in step k , respectively, and let $c_{i,k}$ be the number of prompt tokens allocated to prefill request r_i . The number of tokens scheduled in this step is

$$T_k = \sum_{r_i \in \mathcal{P}_k} c_{i,k} + |\mathcal{D}_k|, \quad T_k \leq B_{\max}, \quad (2)$$

where each decode request contributes one token and $B_{\max}$ is the engine’s maximum per-step token budget. The resulting queuing delay and intervals between decoding iterations determine whether each request meets its TTFT and TPOT objectives.

B. Multi-SLO Traces for Investigations

We construct three representative LLM inference workloads covering conversational assistance, mathematical reasoning, and software-engineering agents.

Chatbots. We construct chatbot requests from LMSYS-Chat-1M, which contains one million real-world conversations collected from interactions with 25 LLMs. The token length of each recorded response determines its target output length.

Mathematical Reasoning. We use all 8,792 questions from the training and test splits of GSM8K. Each word problem serves as the input prompt, while the token length of its reference solution determines the target output length.

Software Agents. We use the software-engineering subset of AgentWorldBench and retain one request from each of 99 distinct trajectories. For each request, we generate 1,024 variants by prepending a distinct synthetic prefix while preserving the original agent action and target environment observation.

From the three request pools, we randomly sample 10,000 chatbot, 8,000 mathematical-reasoning, and 4,000 software-agent requests to form the replay traces.

Arrivals and SLOs. We associate requests in order with timestamps from failure-free entries in BurstGPT and replay each workload according to these timestamps, thereby preserving the trace’s temporal arrival pattern. For each workload, we first determine the sending rate λ_{90} at which vLLM achieves a 90% SLO attainment rate. We then set the sending rate to $1.2\lambda_{90}$ to emulate a workload under pressure.

For each request, we execute it alone on an otherwise idle vLLM deployment and record its measured TTFT and TPOT.

TABLE II
EXPERIMENTAL SETUP.

	Specifications
Hardware	2× Intel Xeon Gold 6530 CPUs, 64 physical cores 128 logical threads, 377.6 GiB host RAM (OS-reported) 4× RTX 6000 Ada GPUs (48 GB each); 2× GPUs used per run
Software	Qwen/Qwen3-14B (TP = 2); vLLM 0.17.0 PyTorch 2.10.0+cu128; CUDA 12.8; driver 570.124.06 LMSYS-Chat-1M, GSM8K, AgentWorldBench-SWE, BurstGPT

TABLE III
SLO ATTAINMENT OF VLLM ON THE MATH WORKLOAD.

SLO Class	TTFT (%)	TPOT (%)	Joint (%)
Overall	75.80	77.68	53.48
TTFT-prioritized	48.89	100.00	48.89
TPOT-prioritized	100.00	12.00	12.00
Balanced	57.39	100.00	57.39
Relaxed	100.00	100.00	100.00

These isolated measurements serve as request-specific latency baselines. We derive the two SLOs of request r_i as

$$\begin{aligned} S_i^{\text{TTFT}} &= c_i^{\text{TTFT}} \widehat{\text{TTFT}}_i, \\ S_i^{\text{TPOT}} &= c_i^{\text{TPOT}} \widehat{\text{TPOT}}_i, \end{aligned} \quad (3)$$

where $\widehat{\text{TTFT}}_i$ and $\widehat{\text{TPOT}}_i$ are the isolated-run measurements, and c_i^{TTFT} and c_i^{TPOT} are the coefficients assigned to the request’s SLO class. Using a fixed random seed, we assign requests equally among four SLO classes: TTFT-prioritized, TPOT-prioritized, balanced, and relaxed. For Qwen3-14B, the TTFT/TPOT coefficient pairs for these four classes are (20, 2.5), (50, 1.5), (25, 2), and (50, 3), respectively. All compared systems use the same request sequence, arrival schedule, and SLO targets under each experimental condition.

C. Low Multi-SLO Serving Performance

We use vLLM’s default FCFS scheduler as a representative serving baseline. It supports continuous batching and chunked prefill, but schedules requests without differentiating their TTFT and TPOT objectives. We run Qwen3-14B with TP = 2 on two GPUs using the configuration in Table II. We use the math workload as a representative case and replay it at its stress load of $1.2\lambda_{90}$.

Table III summarizes the SLO attainment rates for the math workload. Under FCFS, the overall TTFT and TPOT attainment rates are 75.80% and 77.68%, respectively, but only 53.48% of requests satisfy both objectives. The gap between individual and joint attainment shows that satisfying either latency objective alone does not provide effective Multi-SLO service.

The class-level results further reveal distinct failure modes. TTFT-prioritized requests achieve 48.89% TTFT attainment and 100.00% TPOT attainment, whereas TPOT-prioritized requests achieve 100.00% and 12.00%, respectively. Balanced and relaxed requests attain both objectives for 57.39% and 100.00% of their requests, respectively. The contrasting class-level results indicate that the low joint attainment cannot be attributed to a single global latency bottleneck.

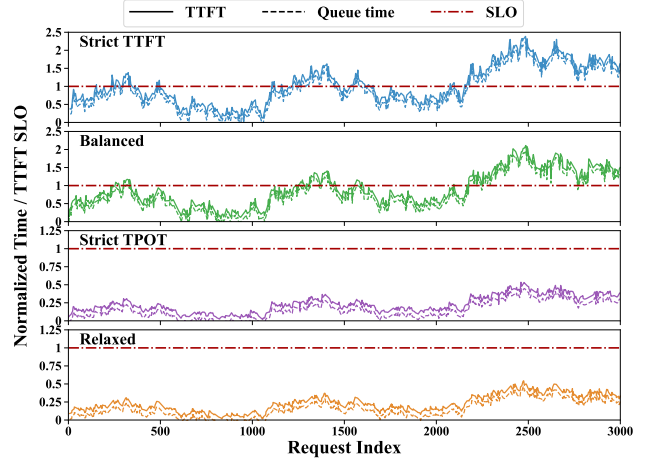


Fig. 2. Normalized TTFT and queueing time under FCFS.

Together, these observations expose a mismatch between throughput-oriented scheduling and request-specific latency objectives. FCFS fixes the service order according to request arrival and does not account for the remaining TTFT or TPOT slack when admitting prefill chunks or advancing decode requests. Consequently, a scheduling decision that preserves prompt responsiveness may disrupt token delivery, while one that protects decode progress may increase the waiting time of queued prefills. The serving engine can therefore retain sufficient capacity to satisfy 100.00% of the relaxed requests, yet fail to direct that capacity toward the phase-specific objectives of other requests. These observations motivate a closer examination of how request ordering and per-step token allocation jointly affect Multi-SLO attainment.

D. Diving into Underlying Reasons

Our investigation identifies two complementary causes of the low Multi-SLO attainment: 1) FCFS distributes queueing delay without considering request-specific TTFT urgency, and 2) a fixed token budget does not provide a reliable latency bound for steps that mix prefill and decode tokens.

1) SLO-unaware temporal scheduling. The first-token latency of request r_i can be decomposed as

$$\text{TTFT}_i = W_i^{\text{queue}} + L_i^{\text{prefill}}, \quad (4)$$

where W_i^{queue} is the delay from arrival until the request is first admitted for prefill, and L_i^{prefill} is the remaining latency until its first token is produced. For a chunked prefill, L_i^{prefill} includes the execution and intervening scheduling delays of all prompt chunks after admission.

Fig. 2 plots TTFT and queueing time for the first 3,001 math requests, normalized by each request’s TTFT SLO. Solid and dashed lines denote TTFT and queueing time, respectively; the dash-dotted line marks the SLO boundary. The Strict TTFT and Strict TPOT panels correspond to the TTFT-prioritized and TPOT-prioritized classes, respectively. The normalized queueing-time curve closely follows the TTFT curve in all

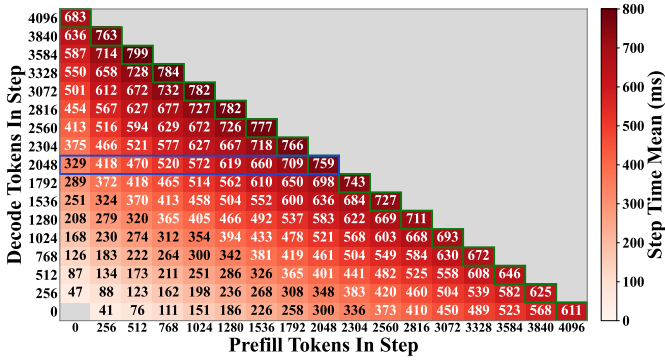


Fig. 3. Mean step time with different prefill and decode token counts.

four SLO classes, indicating that queueing dominates the first-token latency. For TTFT-prioritized and balanced requests, the queueing delay frequently consumes most, or even all, of the TTFT budget before prefill completes. TPOT-prioritized and relaxed requests remain below the boundary because their looser TTFT objectives provide substantially more slack. The resulting TTFT violations therefore arise primarily from how waiting time is distributed among requests, rather than from a uniform shortage of prefill compute capacity.

FCFS creates this imbalance because it admits requests according to arrival order, regardless of their remaining TTFT slack. A TTFT-prioritized request may wait behind earlier requests whose looser objectives could tolerate additional delay; the urgent request exhausts its latency budget in the queue while those ahead of it still finish within their SLOs. The temporal scheduling problem is thus to determine *which request* should receive prefill service first. Accounting for the remaining TTFT slack can redistribute queueing delay toward requests that can tolerate it, without changing the total amount of prefill work.

2) Workload-unaware spatial scheduling. The second failure mode arises after requests enter the running batch. For an active decode request, the interval before its next token is determined by the duration of the engine step in which that token is scheduled. Reordering waiting requests alone cannot protect TPOT because a decode token may still share a step with a large prefill chunk.

To quantify this interference, we vary the numbers of prefill tokens p and decode tokens d scheduled in the same step and measure the mean step time. Each configuration is measured on Qwen3-14B with $TP = 2$ over 50 timed steps (5 rounds $\times$ 10 measurements); gray cells exceed the maximum per-step token budget. As highlighted by the blue outline in Fig. 3, when $d = 2,048$, increasing p from 0 to 2,048 increases the mean step time from 329ms to 759ms. Every active decode request must wait for the longer step to complete before receiving its next token, directly increasing its TPOT.

More importantly, the total token count alone does not determine step time. Along the green boundary where $p + d = 4,096$, the pure-prefill and pure-decode configurations take 611ms and 683ms, respectively, while mixed configurations

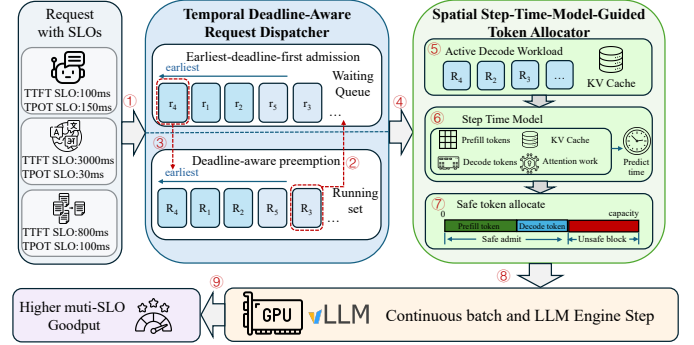


Fig. 4. Design overview of TANGO.

reach up to 799 ms. Thus, the same total token budget produces about a $1.3\times$ difference in latency. This nonlinearity reflects the distinct execution characteristics of compute-intensive prefill and memory-intensive decode, together with their interference when colocated in one iteration. Consequently, enforcing only $p + d \leq B_{\max}$ bounds the amount of scheduled work but cannot reliably bound the wall-clock latency experienced by active decode requests.

The spatial scheduling problem is therefore to determine *how many prefill tokens* can safely share the next step with the current decode workload. This requires an estimate of mixed-step latency and a mechanism that converts the remaining TPOT slack of active requests into a safe prefill allocation. Temporal request ordering and spatial token allocation address different failure modes: optimizing either dimension alone leaves the other source of SLO violations unresolved. These observations motivate their joint design in TANGO.

IV. TANGO METHODOLOGY

Fig. 4 shows the design overview of TANGO, which comprises a deadline-aware request dispatcher and a step-time-model-guided token allocator. At each iteration boundary, the two components coordinate request execution priority and per-step token allocation according to heterogeneous TTFT and TPOT objectives. They take each request's SLOs, inference phase, and execution progress as input and produce a mixed batch of prefill chunks and decode tokens.

The temporal scheduling challenge is to compare request urgency across inference phases. A request with an unfinished prefill must produce its first token within its TTFT budget, whereas a decoding request must sustain token-generation progress within its TPOT budget. The dispatcher maps these phase-specific requirements to comparable deadlines and updates request priorities as inference progresses. These priorities guide the admission of waiting requests and the preemption of less urgent ongoing requests under resource contention (Section IV-A).

The spatial scheduling challenge is to advance urgent prefills while limiting their interference with decoding requests. As shown in Section III-D, the total token count alone cannot reliably determine mixed-step execution time. The allocator therefore uses a step-time model to predict the duration of

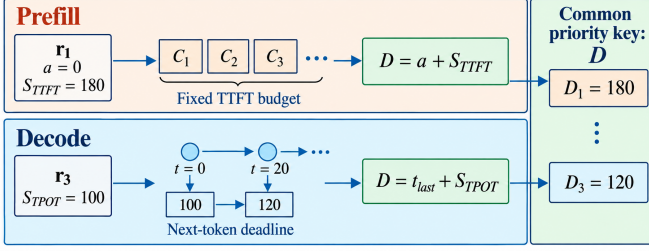


Fig. 5. Phase-aware deadlines for request prioritization (times in ms).

candidate batches from their prefill/decode composition and context-dependent attention work. It converts the remaining deadline slack of selected decode requests into a prefill token budget and distributes that budget according to the dispatcher’s priorities (Section IV-B).

Specifically, TANGO operates as follows. First, the scheduler collects the running requests’ execution state, and the allocator derives a prefill token budget using the step-time model. Second, the dispatcher orders waiting requests by their phase-aware deadlines for admission alongside ongoing requests. If a waiting request cannot obtain KV-cache blocks, the dispatcher considers preempting the running request with the latest deadline, but proceeds only when the waiting request has an earlier deadline. The allocator applies the remaining token and time budgets to the prefill chunks of ongoing and newly admitted requests. Finally, the engine executes the resulting batch and returns token timestamps and request progress to update the next scheduling decision. Unfinished requests remain eligible for reconsideration at the next iteration boundary.

The example in Fig. 4 illustrates how the two components cooperate. An urgent waiting request r_4 receives earlier admission, while a less urgent ongoing request can be preempted to release capacity. The allocator then determines how much of the admitted request’s prompt can share a step with the active decode workload. When the decode requests have little remaining deadline slack, it limits the prefill chunk size and leaves the remaining prompt tokens for subsequent steps, even if the engine’s maximum token capacity has not been exhausted. Thus, request urgency determines the service order, while predicted step time determines how much work can execute in that order.

A. deadline-aware request dispatcher

Figs. 5 and 6 illustrate the dispatcher, which uses phase-aware deadlines as a common priority signal for request admission and preemption. In this section, we first introduce phase-aware deadline construction (Fig. 5), then explain how these deadlines guide waiting-queue ordering and preemption under KV-cache memory pressure (Fig. 6).

1) *Phase-Aware Request Ordering*: As analyzed in Section III-D, arrival order does not reflect the different latency budgets of waiting requests. Moreover, a request’s active latency objective changes during inference: before its first

output token, it must produce that token within its TTFT budget; afterward, it must make timely progress in decoding. Our key idea is to express these phase-specific objectives as an effective deadline on the same time axis. Let q_i denote the number of output tokens already returned by request r_i . Using the timestamps and SLOs defined in Section III-A, its current deadline is

$$D_i = \begin{cases} a_i + S_i^{\text{TTFT}}, & q_i = 0, \\ t_{i,q_i} + S_i^{\text{TPOT}}, & q_i > 0. \end{cases} \quad (5)$$

The first case bounds the time available before the first token, while the second gives a rolling target for the next token based on the most recent output timestamp, denoted by t_{last} in Fig. 5. Both cases map the active latency objective to the common priority key D , allowing the dispatcher to compare requests across inference phases.

The deadline is maintained according to output progress rather than reset whenever a request is scheduled. During chunked prefill, all chunks share the original TTFT deadline, so both queueing and the execution of successive chunks consume the same latency budget. For example, r_1 in Fig. 5 arrives at 0 ms with a TTFT SLO of 180 ms; its deadline remains 180 ms across all prompt chunks. When the first token is returned, the active objective switches to TPOT. Each subsequent output token updates the timestamp in Eq. 5 and establishes a new next-token deadline. For r_3 with a TPOT SLO of 100 ms, advancing the latest output timestamp from 0 to 20 ms moves its deadline from 100 to 120 ms. This rolling deadline provides a local target for each inter-token interval; meeting these targets is sufficient to satisfy the average-TPOT objective in Eq. 1.

At each iteration boundary, the dispatcher orders waiting requests by increasing D_i and admits them in earliest-deadline-first (EDF) order. Arrival time breaks deadline ties, with request identifiers providing deterministic ordering when both values are equal. The waiting queue contains both newly arrived requests and preempted requests awaiting resumption. A request that has not returned its first token is ranked by its TTFT deadline, while a resumed decode request is ranked by its TPOT deadline, even if recovering its context requires additional prefill work. Admission remains subject to the engine’s sequence and memory limits and the token budget supplied by the allocator. In Fig. 6, r_4 with $D_4 = 60$ ms is considered before r_5 with $D_5 = 200$ ms. This ordering allows requests with later deadlines to absorb more queueing delay, giving earlier service to more urgent requests without changing their total prefill work.

2) *Deadline-Aware Preemption*: During continuous batching, a new or resumed request may fail to obtain the KV-cache blocks required for admission. The dispatcher then uses the same deadline representation to identify a running request that can yield memory. For the current running set $\mathcal{R}_k$, it selects the candidate victim as

$$v = \arg \max_{i: r_i \in \mathcal{R}_k} (D_i, a_i), \quad (6)$$

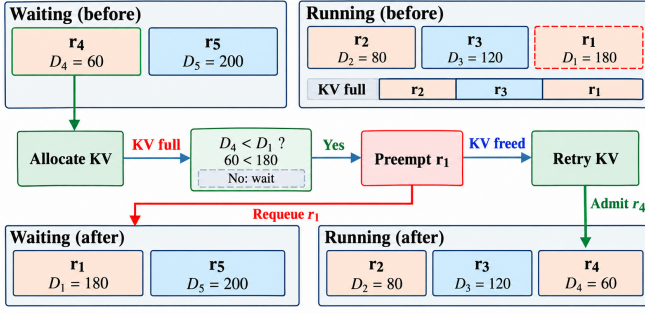


Fig. 6. Deadline-aware admission and preemption (times in ms).

where the tuple is compared lexicographically, so a later-arriving request is selected when deadlines are equal. For waiting request r_j , preemption proceeds only if $D_j < D_v$. Otherwise, r_j remains queued, preserving the running requests whose deadlines are equally or more urgent.

When the comparison permits preemption, the engine reclaims the victim's KV-cache blocks, returns it to the waiting queue, and retries admission of r_j . Preemption occurs at an iteration boundary and preserves the victim's SLOs, arrival time, output-token count, and latest output timestamp. The victim re-enters the queue with the deadline of its current phase: its original TTFT deadline before the first token, or its existing next-token deadline afterward. Thus, preemption does not reset the latency budget, and waiting for resumption continues to consume its remaining slack. In the example at $t = 40$ ms in Fig. 6, KV allocation for r_4 fails, and r_1 has the latest deadline among the running requests. Since $D_4 = 60$ ms is earlier than $D_1 = 180$ ms, the dispatcher preempts r_1 and admits r_4 after retrying allocation. Request r_1 retains its 180 ms deadline and returns to the waiting queue ahead of r_5 , while r_2 and r_3 remain running.

B. step-time-model-guided token allocator

Fig. 7 shows how the allocator determines the amount of prefill work that can share an engine step with the active decode workload. Its key idea is to translate the remaining next-token time budget into a prefill token allocation using a fitted step-time model. In this section, we first introduce the model, then explain how heterogeneous TTFT and TPOT objectives guide budgeting, and finally describe the allocation workflow.

1) *Step-Time Modeling:* As analyzed in Section III-D, batches with the same total token count can have substantially different execution times. The predictor must therefore account for both batch composition and context-dependent attention work. Let p and d denote the numbers of prefill and decode tokens, respectively, K^{dec} the sum of context lengths accessed by decode requests, and A^{pre} the effective attention work of prefill chunks. These four inputs feed the time cost model in Fig. 7, which estimates the step duration as

$$\hat{L}(p, d, K^{\text{dec}}, A^{\text{pre}}) = \alpha + \beta p + \gamma d + \delta K^{\text{dec}} + \epsilon A^{\text{pre}}. \quad (7)$$

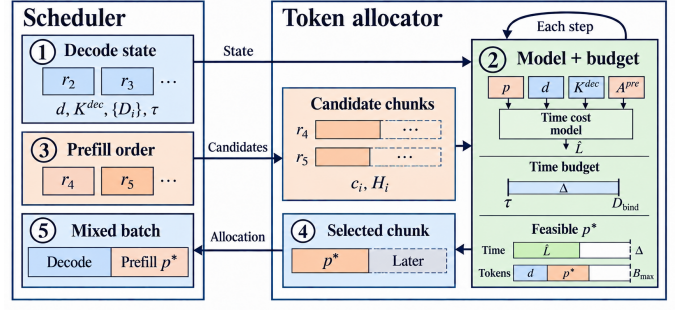


Fig. 7. Execution workflow of the step-time-model-guided token allocator.

Each term approximates a different source of execution cost. The intercept α captures the fixed runtime cost of a step, while βp and γd capture the token-dependent costs of prefill and decode computation. The term δK^{dec} accounts for decode attention's KV-cache reads, which grow with the total history accessed by decode tokens. Finally, ϵA^{pre} accounts for the attention computation performed by prefill chunks.

For chunked prefill, the attention work depends on both the new chunk and its existing prefix. Let $H_{i,k}$ be the already-computed context length available to request r_i before step k . Using the chunk sizes $c_{i,k}$ defined in Eq. 2, we approximate the attention work as

$$A_k^{\text{pre}} = \sum_{r_i \in \mathcal{P}_k} c_{i,k} \left(H_{i,k} + \frac{c_{i,k}}{2} \right). \quad (8)$$

A single fresh prefill thus contributes approximately $p^2/2$, whereas a chunk following a long prefix incurs additional work proportional to that prefix length. Consequently, equal-sized chunks can consume different amounts of the step-time budget. The coefficients are fitted from profiled engine steps for the deployed model and hardware configuration. At runtime, prediction requires only these four workload statistics and a few arithmetic operations.

2) *Joint TTFT/TPOT Budgeting:* At the beginning of step k , the allocator summarizes the active decode workload as $d_k = |\mathcal{D}_k|$ and $K_k^{\text{dec}} = \sum_{r_i \in \mathcal{D}_k} H_{i,k}$. Each decode request contributes its next-token deadline D_i from Eq. 5. The allocator selects a binding deadline from this set using a configurable percentile $\rho_0 \in [0, 1]$. With $\rho_0 = 0$, the earliest deadline bounds the predicted step duration; a larger percentile selects a later deadline and permits more prefill work.

A fixed decode percentile can leave urgent prefills with insufficient service even when increasing prefill work would improve first-token responsiveness. To account for this trade-off, the allocator compares the remaining fractions of the two latency budgets. At scheduling time τ_k , these fractions are

$$u_{i,k}^{\text{TTFT}} = \frac{D_i - \tau_k}{S_i^{\text{TTFT}}}, \quad q_i = 0, \\ u_{i,k}^{\text{TPOT}} = \frac{D_i - \tau_k}{S_i^{\text{TPOT}}}, \quad q_i > 0. \quad (9)$$

The allocator compares the minimum TTFT fraction among requests awaiting their first token in the running set and waiting queue with the minimum TPOT fraction among active decode requests. If TTFT is more urgent, it moves the active percentile ρ_k from ρ_0 toward 1, allowing a later decode deadline to release additional prefill capacity. Otherwise, it retains $\rho_k = \rho_0$. This adaptation gives urgent prefills more service by relaxing the step-time target imposed by earlier decode deadlines.

Let Quantile_{ρ_k} select a deadline at percentile ρ_k from the ordered decode deadlines. Denote the selected deadline by D_k^{bind} , shown as D_{bind} in Fig. 7. The time available for the next step is

$$\begin{aligned} D_k^{\text{bind}} &= \text{Quantile}_{\rho_k}(\{D_i : r_i \in \mathcal{D}_k\}), \\ \Delta_k &= D_k^{\text{bind}} - \tau_k. \end{aligned} \quad (10)$$

The full prediction in Eq. 7 is compared with Δ_k , since fixed and decode-related costs are already included in the model. If no active decode request contributes a TPOT deadline, the allocator uses the remaining engine token capacity without an additional TPOT constraint. If $\Delta_k \leq 0$, it admits no prefill tokens in this step.

For a positive time budget, the allocator seeks the largest feasible prefill allocation:

$$\begin{aligned} p_k^* &= \max_{p \in \mathbb{Z}_{\geq 0}} p, \\ \text{s.t. } p + d_k &\leq B_{\text{max}}, \\ \widehat{L}(p, d_k, K_k^{\text{dec}}, A_k^{\text{pre}}(p)) &\leq \Delta_k. \end{aligned} \quad (11)$$

Here, $A_k^{\text{pre}}(p)$ is computed from the candidate chunks in their scheduling order using Eq. 8. Once the decode workload and candidate order are fixed, only the admitted prefill token count varies, making the budget search one-dimensional. If even a decode-only step exceeds the selected time budget, the allocator sets the additional prefill allocation to zero and allows decode requests to continue making progress. The inequality provides a model-based scheduling target whose accuracy depends on the fitted predictor.

3) *Token Allocation Workflow*: Fig. 7 illustrates the interaction between the scheduler and allocator. At each iteration boundary, the scheduler collects the active decode workload, next-token deadlines, and current time (①). The allocator uses this state to establish the time budget and evaluate the step-time model as prefill candidates are considered (②). The scheduler processes ongoing requests and presents waiting prefill candidates in the order determined by the dispatcher, together with their chunk lengths and existing context lengths (③). For each candidate, the allocator combines these prefill statistics with the decode workload and checks both constraints in Eq. 11 to select a feasible chunk (④). The selected prefill tokens join the active decode tokens to form the mixed batch (⑤).

Running prefill chunks and newly admitted prefills share the same budget. As candidates are allocated, the allocator updates both the cumulative prefill token count and attention work. If the full chunk does not fit, it truncates the chunk to the

largest feasible size; if no positive allocation remains, further prefill work waits for a subsequent step. The dashed segments in Fig. 7 represent this deferred work. Decode tokens remain scheduled subject to the engine’s resource limits.

In the example in Fig. 7, r_2 and r_3 remain active decode requests, while r_4 and r_5 are considered for prefill in that order. The allocator selects a feasible chunk from r_4 ’s prompt, leaving the remaining prefill work for later iterations. Thus, earlier admission does not require an entire prompt to fit in one step. After execution, updated token timestamps and context lengths determine the next allocation.

V. IMPLEMENTATION OF TANGO

TANGO is implemented on top of vLLM 0.17.0 by extending its request admission path and scheduler. The dispatcher and allocator run in the CPU-side scheduling loop and produce per-request token allocations for each engine step. The resulting batch is passed to vLLM through its existing scheduler-executor interface, reusing continuous batching, chunked prefill, PagedAttention-based KV-cache management, tensor parallelism, sampling, and streaming output.

SLO Metadata and Runtime State: We extend request admission to carry two optional SLO fields, TTFT and TPOT, in milliseconds. A missing SLO is treated as an absent constraint, allowing best-effort requests to use the same serving path. Alongside these fields, the scheduler maintains each request’s arrival timestamp, prompt length, computed-token progress, output-token count, and latest output timestamp in host memory. These fields are updated together with vLLM’s request state after each engine step. The first emitted token activates the TPOT objective, and subsequent token timestamps advance the next-token deadline.

Queue and Preemption Hooks: We add queue and preemption hooks to vLLM’s waiting-request admission path. The queue hook applies the phase-aware priority and tie-breaking rules in Section IV-A. On KV-cache allocation failure, the preemption hook selects the running request with the latest deadline and requires the waiting request’s deadline to be strictly earlier. If permitted, vLLM reclaims the victim’s KV blocks, moves it from RUNNING to WAITING, and retries allocation. The victim keeps its SLOs, output progress, and the timestamps used to construct its deadline.

Per-Step Token Allocation: We fit the step-time model coefficients from offline profiling for the deployed model and hardware configuration. At each iteration boundary, the allocator summarizes the active decode workload and collects its next-token deadlines from scheduler-maintained state. It then establishes the time budget using the percentile policy in Section IV-B. As the scheduler processes ongoing prefill requests and admits waiting requests, their chunks share this budget and the engine’s maximum per-step token capacity. The allocator updates the cumulative prefill token count and attention work after each allocation, using the candidate’s chunk length and already-computed context length. A chunk that exceeds either limit is truncated to its largest feasible size; unallocated tokens remain for later steps. The resulting token

allocations are forwarded to the executor through the standard scheduling output.

VI. EVALUATION OF TANGO

VII. CONCLUSION